BP101T COMPANION TEXT

Python for Pharmacists

Unit II · Control Structures & Functions

BioDockify

Building the Future of AI-Driven Pharmaceutical Research

learn.biodockify.com

Course BP101T · B.Pharm Semester I

Chapters 6–8 · Six teaching hours

Theory-first · 58 code examples, outputs machine-verified · modular dosage & BMI programs

Basics of Python Programming for Pharmaceutical Sciences

Contents

Chapter 6 — Conditional Statements....1
6.1 Why programs must decide....1
6.2 The if statement — theory and anatomy....2
6.3 if-else — two roads....3
6.4 if-elif-else — many roads, first True wins....5
6.5 Nested conditions — a question inside a question....6
Chapter 7 — Loops....8
7.1 Why loops exist....8
7.2 The for loop — anatomy and the loop variable....8
7.3 range() — counting without a list....9
7.4 The accumulator and counter patterns....10
7.5 The while loop — repeat while a condition holds....11
7.6 break and continue — the syllabus pair....13
Chapter 8 — Functions....15
8.1 Why functions exist....15
8.2 Anatomy of a function....15
8.3 Parameters and arguments....15
8.4 return — sending the answer back....16
8.5 Local scope — a function's variables are private....17
8.6 Modular program 1 — dosage calculation (syllabus)....18
8.7 Modular program 2 — BMI calculation (syllabus)....18
8.8 The dispensary toolkit — a third assembler....19
Unit II — Consolidation....19
Syllabus coverage map....20
Twenty-question exercise bank....20
Answers to selected exercises....23
Important points for examination....23

Right-click the table of contents and choose “Update Field” to refresh page numbers.

Unit hours: 6 · **Book chapters:** 6–8 · **Syllabus bullets covered:** conditional statements (if, if-else, if-elif-else) and nested conditions; loops (for loop, while loop); break and continue statements; defining and calling functions, passing arguments and returning values; writing modular programs for simple pharmaceutical applications — dosage calculation and BMI calculation.

Unit objectives. After studying this unit you should be able to: (1) write programs that branch using if, if-else and if-elif-else; (2) place one decision inside another (nesting) and read nested code confidently; (3) repeat work with for and while loops; (4) exit or skip loop rounds with break and continue; (5) define functions with parameters and return values; and (6) assemble modular pharmacy programs for dosage and BMI calculation.

Chapter 6 — Conditional Statements

6.1 Why programs must decide

Every program written so far runs the same lines in the same order, every time. Real pharmacy work is not like that: a dose above the maximum must raise an alert and a dose within limits must not; a batch either clears quality control or it does not. A **conditional statement** tests a condition and lets the program choose which block of code to carry out — a Boolean expression that evaluates to `True` or `False`.

The decision vocabulary from Unit I already supplies the conditions: comparisons (`dose > 500`, `age <= 12`) and logic (`in_stock and not expired`). This chapter adds the *grammar* that acts on them.

6.2 The if statement — theory and anatomy

`if` executes its guarded block **only** on a True condition; otherwise the block is skipped and the program continues below it. Three syntax rules are absolute:

1. The condition line ends with a **colon**.
2. The guarded lines form an indented block underneath (four spaces; Jupyter adds them automatically after the colon).
3. **Indentation is not decoration in Python** — it *is* the block structure. Python uses indentation instead of curly braces; wrong indentation raises an `IndentationError`, and lines back at the margin belong to the surrounding program, not the `if`.

```
dose = 600
if dose > 500:
    print("High dose - check with the prescriber")
if dose > 0:
    print("Dose recorded")
print("Check complete")
```

OUTPUT

```
High dose - check with the prescriber
Dose recorded
Check complete
```

Read it aloud: *if* the dose exceeds 500 mg, warn; *if* it is positive, record it; *always* print "Check complete" (no indent — no guard). Each `if` is tested independently.

Five examples — independent guards:

```
temp_c = 36.9
if temp_c > 37.5:
    print("Fever detected")
print("Reading done")            # runs - it is outside the if
```

OUTPUT

```
Reading done
```

```
stock = 10
if stock < 20:
    print("LOW STOCK - reorder")
```

OUTPUT

```
LOW STOCK - reorder
```

```
expired = False
if expired:
    print("Quarantine this batch")
# nothing prints: False skips the block
```

```
if "statin" in "Atorvastatin":
    print("Lipid-lowering family")
```

OUTPUT

```
Lipid-lowering family
```

```
rx_ok, in_stock, expired = True, True, False
if rx_ok and in_stock and not expired:
    print("Dispense")
```

OUTPUT

```
Dispense
```

The last example is the pharmacy's dispensing checklist switched on: one line of decision logic replacing three manual checks.

6.3 if-else — two roads

`if-else` guarantees exactly one of two blocks runs: the first when the condition is True, the `else` block in every other case. The `else` line takes **no condition** — it is the catch-all. Picture a fork in the road: the condition is the signpost, and both roads rejoin afterwards.

```
temperature = 39.5
if temperature > 38:
    print("Fever detected")
else:
    print("Temperature normal")
```

OUTPUT

```
Fever detected
```

Five examples, with the boundary lesson embedded:

```
marks = 50
if marks >= 50:
    print("PASS")
else:
    print("FAIL - reattempt needed")
```

OUTPUT

```
PASS
```

```
marks = 49.9
if marks >= 50:
    print("PASS")
else:
    print("FAIL - reattempt needed")
```

OUTPUT

```
FAIL - reattempt needed
```

Q1 vs Q2 — one-tenth of a mark apart, opposite outcomes: `>=` **includes** the boundary value itself, `>` excludes it. On pass marks and fever thresholds, the choice of operator is policy, not style.

```
dose, max_dose = 650, 500.0
if dose > max_dose:
    print("Alert: exceeds the safe maximum")
else:
    print("Within the safe range")
```

OUTPUT

```
Alert: exceeds the safe maximum
```

```
bp = 118
if bp < 90:
    print("Low BP")
else:
    print("Acceptable BP")
```

OUTPUT

```
Acceptable BP
```

```
sex = "F"
if sex == "F":
    print("CrCl correction factor 0.85 applies")
else:
    print("No correction factor")
```

OUTPUT

```
CrCl correction factor 0.85 applies
```

6.4 if-elif-else — many roads, first True wins

When several mutually exclusive cases exist, chain them with `elif` (short for *else if*). Python evaluates the conditions **top to bottom** and runs the block of the **first** True condition; every later condition is skipped without being checked. A trailing `else` catches "none of the above".

```
marks = 72
if marks >= 70:
    grade = "Distinction"
elif marks >= 60:
    grade = "First Class"
elif marks >= 50:
    grade = "Pass"
else:
    grade = "Fail"
print(f"Grade: {grade}")
```

OUTPUT

```
Grade: Distinction
```

Trace ritual (an examination skill). For `marks = 65`, write T/F beside each condition: `65 >= 70` F (skip), `65 >= 60` **T** (run, then stop) — the `>= 50` test below is never even evaluated. For `marks = 55`: F, F, T → Pass. For `marks = 40`: F, F, F → the `else` net catches it → Fail.

Order matters — the silent bug. Put the loosest test first and it swallows everything:

```
marks = 88
if marks >= 50:
    grade = "Pass"            # 88 lands here and stops!
elif marks >= 60:
    grade = "First Class"     # dead code
elif marks >= 70:
    grade = "Distinction"     # unreachable
print(grade)
```

OUTPUT

```
Pass
```

No error message appears — the program simply answers wrongly. Rule for the notebook: **order conditions from the strictest to the loosest** (highest bar first, catch-all last).

Five examples:

```
bp = 145
if bp < 90:
    print("Low BP")
elif bp < 120:
    print("Normal BP")
elif bp < 140:
    print("Elevated BP")
else:
    print("High BP - consult doctor")
```

OUTPUT

```
High BP - consult doctor
```

```
age = 8
if age < 12:
    band = "Paediatric - weight-based dosing"
elif age < 65:
    band = "Adult - standard dosing"
else:
    band = "Geriatric - start low, go slow"
print(band)
```

OUTPUT

```
Paediatric - weight-based dosing
```

```
bmi = 24.2
if bmi < 18.5:
    category = "Low weight"
elif bmi < 25:
    category = "Normal"
elif bmi < 30:
    category = "Overweight"
else:
    category = "Obese"
print(category)
```

OUTPUT

```
Normal
```

```
severity = "moderate"
if severity == "mild":
    action = "Document and observe"
elif severity == "moderate":
    action = "Monitor closely, consider dechallenge"
elif severity == "severe":
    action = "Stop the drug, notify the physician"
else:
```

```
    action = "Unknown grade - review the report"
print(action)
```

OUTPUT

```
Monitor closely, consider dechallenge
```

```
scr = 2.8
if scr < 1.2: level = "Normal creatinine"
elif scr < 2.0: level = "Mildly raised"
else: level = "Markedly raised - renal review"
print(level)
```

OUTPUT

```
Markedly raised - renal review
```

Note the ADR example's `else` net: an unrecognized severity (a typo like `"Medium"`) lands safely in "Unknown grade" instead of being silently ignored — in pharmacovigilance, unhandled categories must never vanish.

6.5 Nested conditions — a question inside a question

A **nested condition** means an `if` sits inside the block of a further `if`. Use it when a second question only makes sense after the first is answered: the pharmacist verifies the prescription first and only then checks stock. Indentation deepens by four spaces per level — three levels (twelve spaces) is the practical limit before the logic should be reorganised.

```
rx_ok = True
in_stock = True
expired = False

if rx_ok:
    if in_stock:
        if not expired:
            print("DISPENSE")
        else:
            print("QUARANTINE - expired stock")
    else:
        print("OUT OF STOCK - reorder")
else:
    print("REFUSE - prescription required")
```

OUTPUT

```
DISPENSE
```

The funnel: each gate closes before deeper questions are asked. Flip `expired` to `True` and the quarantine branch fires without touching stock or prescription logic.

Nest or combine? If every branch needs a *different* message, nest. If one action covers a combination, combine with `and`: `if rx_ok and in_stock and not expired: print("DISPENSE")` — flatter, but with no room for separate replies.

The validate-first pattern. The most valuable nesting idiom in this course checks that input is usable *before* acting on it:

```
marks = 105
if marks < 0 or marks > 100:
    print("Invalid marks - enter 0 to 100")
else:
    if marks >= 50:
        print("PASS")
    else:
        print("FAIL")
```

OUTPUT

```
Invalid marks - enter 0 to 100
```

Invalid data gets one message and nothing else runs — the grade logic lives inside `else`, guaranteed to see only valid marks.

Age-and-weight dosing tree (symmetric nesting, four leaves):

```
age, weight = 8, 15
if age < 12:
    if weight < 20:
        dose = 100      # small child
    else:
        dose = 200      # older child
else:
    if age >= 65:
        dose = 400      # cautious senior dose
    else:
        dose = 500      # standard adult
print(f"Dose: {dose} mg")
```

OUTPUT

```
Dose: 100 mg
```

Hospital dosing protocols are written as exactly this kind of tree; a student who can trace it can read a guideline.

Chapter summary. `if` guards a block behind a True condition (colon + indent, no exceptions). `if-else` runs exactly one of two roads; the boundary rule (`>=` includes, `>` excludes) turns one character into policy. `elif` chains test top-down, first True wins, strictest-first ordering prevents silent wrong answers, and `else` is the safety net. Nesting puts a question inside a question — validate first, act second.

Quick checks 6. (a) What two things end an `if` line? (b) Output of the BP chain with `bp = 135`? (c) Why does `marks = 88` grade as "Pass" in the buggy chain? (d) When would you nest instead of using `and`? (e) Rewrite the dispensing gate so each failure has its own message.

Chapter 7 — Loops

7.1 Why loops exist

Typing `print("Check: Asha")` forty times for forty patients is not programming. A **loop** runs one block of code again and again — once per patient, per drug, per reading — while the program itself stays the same size. Python offers two: `for` (one pass per item of a known collection) and `while` (repeat as long as a condition holds).

7.2 The for loop — anatomy and the loop variable

```
patients = ["Asha", "Ravi", "Meena"]
for patient in patients:
    print("Checking:", patient)
print("Ward round complete")
```

OUTPUT

```
Checking: Asha
Checking: Ravi
Checking: Meena
Ward round complete
```

Anatomy: `for` and `in` are keywords; `patient` is the **loop variable** — a fresh box that Python fills with the next item before each pass; the collection follows `in`; the indented body runs once per item; lines back at the margin run once, after the loop finishes.

Trace table for the run above:

pass	patient holds	body prints
1	"Asha"	Checking: Asha
2	"Ravi"	Checking: Ravi
3	"Meena"	Checking: Meena
after	loop exhausted	Ward round complete

Any sequence can be walked. A string is a sequence of characters (Unit I's bead-on-a-thread model):

```
for letter in "ASPIRIN":
    print(letter)
```

OUTPUT

```
A
S
P
I
R
I
```

```
N
```

7.3 range() — counting without a list

`range()` manufactures a sequence of numbers. It counts **from zero** and **excludes the stop value** — the same two conventions as string slicing.

Form	Produces
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8 (step 2)

Five examples:

```
for i in range(5):
    print("Ward round", i)
```

OUTPUT

```
Ward round 0
Ward round 1
Ward round 2
Ward round 3
Ward round 4
```

```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
```

OUTPUT

```
10
```

```
for t in range(36, 40):
    print(t, "C")
```

OUTPUT

```
36 C
37 C
38 C
39 C
```

```
for n in range(10, 0, -2):
    print(n)
```

OUTPUT

```
10
8
6
```

```
4
2
```

```
for strip in range(1, 4):
    print("Blister strip", strip, "checked")
```

OUTPUT

```
Blister strip 1 checked
Blister strip 2 checked
Blister strip 3 checked
```

7.4 The accumulator and counter patterns

Two patterns power every calculation over a collection. **Accumulate**: start a total at zero *before* the loop, add each item *inside*, use the result *after*. **Count**: the same skeleton, but add 1 only when a condition holds.

```
doses = [500, 250, 500, 1000, 250]
total = 0
for dose in doses:
    total = total + dose
print(total)
```

OUTPUT

```
2500
```

```
temps = [36.5, 37.8, 38.2, 36.9, 37.1]
total = 0
for t in temps:
    total = total + t
print(f"Ward average: {total / len(temps):.1f} C")
```

OUTPUT

```
Ward average: 37.3 C
```

```
statuses = [False, True, False, True, True]
expired = 0
for status in statuses:
    if status:
        expired = expired + 1
print(expired, "expired batch(es)")
```

OUTPUT

```
3 expired batch(es)
```

```
temps = [36.5, 37.8, 38.2, 36.9, 38.0]
fevers = 0
for t in temps:
    if t > 37.5:
```

```
        print("Fever:", t)
        fevers = fevers + 1
print(f"{fevers} fever case(s)")
```

OUTPUT

```
Fever: 37.8
Fever: 38.2
Fever: 38.0
3 fever case(s)
```

```
items = ["Paracetamol", "Aspirin", "Cetirizine"]
qtys = [45, 8, 60]
shortages = 0
for i in range(3):
    if qtys[i] < 10:
        print("REORDER:", items[i], "-", qtys[i])
        shortages = shortages + 1
print(shortages, "item(s) below minimum")
```

OUTPUT

```
REORDER: Aspirin - 8
1 item(s) below minimum
```

The last example introduces the index walk: `range(3)` produces 0, 1, 2 and `items[i]`/`qtys[i]` pick matching pairs — two lists walked in step (Unit III replaces this with dictionaries and tables).

7.5 The while loop — repeat while a condition holds

`while` re-checks its condition **before every pass** and repeats as long as it stays True. Two theoretical duties follow: the body must contain an **update** that moves the situation toward False; and because the check comes first, a condition that is False initially means the body runs **zero** times.

```
stock = 3
while stock > 0:
    print("Dispensing 1 strip; remaining:", stock - 1)
    stock = stock - 1
print("Out of stock!")
```

OUTPUT

```
Dispensing 1 strip; remaining: 2
Dispensing 1 strip; remaining: 1
Dispensing 1 strip; remaining: 0
Out of stock!
```

Forget the update line and the condition never changes — the **infinite loop**. It is not dangerous (press `Ctrl+C`, or *Interrupt Kernel* in Jupyter) but it is always a bug: every `while` body must answer the question *"which line moves toward the exit?"*

for or while? If the repeats can be counted in advance ("each patient", "five times"), use `for`. If only a condition decides ("while stock remains", "until the user types done"), use `while`.

Five examples:

```
tablets, daily = 31, 2
days = 0
while tablets >= daily:
    tablets = tablets - daily
    days = days + 1
print(f"{days} full days, {tablets} tablet(s) left")
```

OUTPUT

```
15 full days, 1 tablet(s) left
```

```
count = 5
while count > 0:
    print(count)
    count = count - 1
```

OUTPUT

```
5
4
3
2
1
```

```
x = 1
while x < 100:
    x = x * 3
print(x)
```

OUTPUT

```
243
```

```
stock = 0
while stock > 0:
    print("never runs")
print("checked first: body skipped")
```

OUTPUT

```
checked first: body skipped
```

```
dose = 1000
while dose > 120:
    dose = dose // 2        # halving until a safe region
print(dose, "mg")
```

OUTPUT

```
62 mg
```

7.6 break and continue — the syllabus pair

`break` exits the whole loop immediately — remaining items are never touched. `continue` abandons only the current round and jumps to the next item. Mnemonic: **break breaks out; continue carries on.**

break — stop at the first problem. Scanning batch labels, the first expired batch is a quarantine event; scanning further is pointless:

```
batches = ["B1", "B2", "EXPIRED", "B4"]
for b in batches:
    if b == "EXPIRED":
        print("STOP - expired batch found")
        break
    print("Checking batch:", b)
```

OUTPUT

```
Checking batch: B1
Checking batch: B2
STOP - expired batch found
```

B4 is never examined — break means stop.

continue — skip the bad, keep the good. Sensor readings of −1 mark errors; ignore them, keep the rest:

```
temps = [36.5, -1, 37.9, -1, 38.1]
for t in temps:
    if t < 0:
        continue
    print("Read:", t)
```

OUTPUT

```
Read: 36.5
Read: 37.9
Read: 38.1
```

More contrasts:

```
for i in range(4):
    if i == 2:
        break
    print(i)
```

OUTPUT

```
0
1
```

```
for i in range(4):
    if i == 2:
```

```
        continue
    print(i)
```

OUTPUT

```
0
1
3
```

```
for n in range(10, 15):
    if n % 2 == 0:      # even batch number
        continue        # audit odd-numbered batches only
    print("Auditing batch", n)
```

OUTPUT

```
Auditing batch 11
Auditing batch 13
```

The sentinel loop. `while True` looks infinite; `break` is its designed exit door. This eight-line shape is the standard data-entry tool of the unit:

```
total = 0
while True:
    entry = input("Dose in mg (or 'done'): ")
    if entry == "done":
        break
    total = total + float(entry)
print(f"Total: {total} mg")
```

OUTPUT

```
Dose in mg (or 'done'): 500
Dose in mg (or 'done'): 250
Dose in mg (or 'done'): done
Total: 750.0 mg
```

`"done"` is the **sentinel** — the magic word that means stop (it is never added). Read the body top-to-bottom: ask; if done, leave; otherwise add and ask again.

Chapter summary. `for` walks a known collection once per item, the loop variable refreshing each pass; `range()` counts for you (from 0, stop excluded, optional step). Accumulator/counter patterns (`init before, add inside, use after`) turn collections into statistics. `while` repeats on a condition — checked first, updated inside, `Ctrl+C` as the fire exit when the update is forgotten. `break` abandons the loop, `continue` skips one round, and the sentinel loop (`while True` + `break`) is the unit's data-entry workhorse.

Quick checks 7. (a) How many times does `range(3)` repeat its body, and what numbers? (b) In the days-supply loop, which line is the update? (c) Difference between break and continue in one sentence. (d) Predict: `total` after `for x in [2, 4, 6]: total += x`. (e) Why must the sentinel string never be converted with `float()`?

Chapter 8 — Functions

8.1 Why functions exist

Copy-pasting the BMI formula into five notebooks means five places to fix when the formula changes. A **function** is a named, reusable block of code that performs one task: define it once, call it by name forever. The benefits are the syllabus's own words — *reusability* (a single definition serves unlimited calls), *modularity* (large programs become small, single-purpose pieces), and *easier debugging* (each piece can be tested separately). Every calculator written so far in this book becomes a one-line tool by the end of this chapter.

8.2 Anatomy of a function

```
def greet_patient():
    """Print a welcome message for the pharmacy counter."""
    print("Welcome to the pharmacy!")

greet_patient()
```

OUTPUT

```
Welcome to the pharmacy!
```

Four parts: `def` introduces the definition; the **function name** follows (snake_case, same naming rules as variables); **parameters** — the inputs the function accepts — sit in brackets (here, none); the indented body contains the work, optionally opened by a **docstring**, a triple-quoted note describing the function (Unit I's comment rules apply). Defining a function does nothing by itself; **calling** it — name plus brackets — runs it.

8.3 Parameters and arguments

A **parameter** is the placeholder variable in the definition; an **argument** is the concrete value handed over when the call happens. Inside the function, parameters behave like ordinary variables — pre-filled boxes:

```
def calculate_dose(weight, dose_per_kg):    # parameters
    total = weight * dose_per_kg
    return total

result = calculate_dose(60, 5)              # arguments
print("Total dose:", result, "mg")
```

OUTPUT

```
Total dose: 300 mg
```

8.4 return — sending the answer back

`return` delivers a value from inside the function back to its caller, where it can be stored, reused, or fed into another calculation. Without `return`, a function returns the special value `None` — and its work evaporates when it ends.

	`return`	`print()`
Purpose	send a value back to the caller	display text on screen
Result reusable?	yes — store it in a variable	no — the value is lost after display

```
def mg_per_kg(dose, weight):
    return dose / weight

r = mg_per_kg(500, 70)
print(f"{r:.2f} mg per kg")
print(f"Doubled: {mg_per_kg(1000, 70):.2f} mg per kg")
```

OUTPUT

```
7.14 mg per kg
Doubled: 14.29 mg per kg
```

Five examples of returning values:

```
def area_label(w, h):
    return f"{w * h} sq cm"

print(area_label(5, 4))
```

OUTPUT

```
20 sq cm
```

```
def is_fever(temp):
    return temp > 37.5

print(is_fever(38.2), is_fever(36.9))
```

OUTPUT

```
True False
```

```
def no_return():
    x = 42

print(no_return())
```

OUTPUT

```
None
```

```
def strip_count(total, per_strip=10):        # default parameter
    return total // per_strip
```

```
print(strip_count(43))
print(strip_count(43, per_strip=15))         # keyword argument
```

OUTPUT

```
4
2
```

```
def total_dose(*doses):                      # variable-length: any number of
args
    return sum(doses)

print(total_dose(500, 250, 750))
print(total_dose(100))
```

OUTPUT

```
1500
100
```

The last three examples introduce the **argument types** the syllabus names: *positional* (matched by order — all earlier calls), *default* (`per_strip=10` supplies a value when the caller omits it), *keyword* (`per_strip=15` names its target, order no longer matters), and *variable-length* (`*doses` gathers any number of arguments into one collection).

8.5 Local scope — a function's variables are private

Variables created inside a function exist only inside it. This **local scope** is a feature: functions cannot accidentally overwrite the caller's variables.

```
def report():
    dose = 500               # local: exists only inside report()

report()
print(dose + 1)
```

OUTPUT

```
NameError: name 'dose' is not defined
```

(A *beyond the exam* note: a function can call itself — **recursion**; it powers elegant mathematics but is unnecessary for pharmacy programming, and `while` handles those cases more plainly.)

8.6 Modular program 1 — dosage calculation (syllabus)

The syllabus asks for a modular dosage calculator. Three functions, each doing one job: a rate decision (if-elif-else from Chapter 6), a multiplication (Unit I arithmetic), and a report. A loop over a patient list then runs the whole ward through them:

```
def get_dose_rate(age):
    """Weight-based rate in mg/kg by age group."""
```

```
    if age < 12:
        return 8            # children
    elif age < 65:
        return 12           # adults
    else:
        return 6            # elderly - reduced rate

def calculate_dose(weight, rate):
    return weight * rate

def dosage_report(name, age, weight):
    rate = get_dose_rate(age)
    dose = calculate_dose(weight, rate)
    print(f"{name} | Age {age} | {weight} kg -> {dose} mg")

patients = [("Aarav", 8, 22), ("Meera", 35, 50), ("Ravi", 68, 55)]
for name, age, weight in patients:
    dosage_report(name, age, weight)
```

OUTPUT

```
Aarav | Age 8 | 22 kg -> 176 mg
Meera | Age 35 | 50 kg -> 600 mg
Ravi | Age 68 | 55 kg -> 330 mg
```

Note how the loop unpacks each tuple directly into `name, age, weight`, and how `dosage_report` *calls* the other two functions — assembly, not copy-paste. Change the adult rate to 10 and all three reports update: one definition, many uses.

8.7 Modular program 2 — BMI calculation (syllabus)

The same architecture for the BMI calculator: one function computes, one classifies (the Chapter 6 chain), one reports:

```
def calculate_bmi(weight, height):
    """BMI = weight (kg) / height (m) squared."""
    return weight / (height ** 2)

def classify_bmi(bmi):
    if bmi < 18.5:
        return "Low weight"
    elif bmi < 25:
        return "Normal"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

def bmi_report(name, weight, height):
```

```
    bmi = calculate_bmi(weight, height)
    category = classify_bmi(bmi)
    print(f"{name}: BMI {bmi:.1f} -> {category}")

bmi_report("Anita", 55, 1.62)
bmi_report("Vikram", 88, 1.70)
```

OUTPUT

```
Anita: BMI 21.0 -> Normal
Vikram: BMI 30.4 -> Obese
```

Why modular? the two small functions slot straight into other tools — a diet-advice program, a ward screening sheet, a Unit IV dataframe — with the logic written once and never retyped. That is the entire point of functions.

8.8 The dispensary toolkit — a third assembler

To close the unit, the Cockcroft–Gault calculator from Unit I becomes a function and joins the toolkit:

```
def crcl(age, weight, scr, sex="M"):
    """Creatinine clearance (Cockcroft-Gault), mL/min."""
    factor = 0.85 if sex == "F" else 1.0
    return (140 - age) * weight / (72 * scr) * factor

print(f"{crcl(55, 70, 1.0):.1f} mL/min")
print(f"{crcl(55, 70, 1.0, sex='F'):.1f} mL/min")
```

OUTPUT

```
82.6 mL/min
70.2 mL/min
```

One definition, keyword-flexible, unit-tested against the hand calculation — this is what "modular program" means on the examination paper.

Chapter summary. `def name(parameters):` defines a reusable block; calling it runs it. Arguments fill parameters — positionally, by keyword, with defaults, or gathered by `*args`. `return` delivers the answer for storing and reuse (`print` merely displays; no `return` means `None`). Function variables are local. The two syllabus programs — dosage by age group and BMI with classification — demonstrate the modular style: small single-job functions, assembled by a caller.

Quick checks 8. (a) Parameter vs argument, in one sentence each. (b) What does a function without `return` give back? (c) In `strip_count(43, per_strip=15)`, which part is a keyword argument? (d) Why is `is_fever` returning a Boolean more useful than printing "fever"? (e) Name the three functions of the BMI program and their single jobs.

Unit II — Consolidation

Syllabus coverage map

Syllabus bullet	Where
Conditional statements (if, if-else, if-elif-else), nested conditions	Ch 6
Loops (for loop, while loop)	Ch 7 (§7.2–7.5)
break and continue statements	Ch 7 (§7.6)
Defining and calling functions; arguments and return values	Ch 8 (§8.1–8.5)
Modular programs: dosage calculation and BMI calculation	Ch 8 (§8.6–8.8)

Twenty-question exercise bank

Predict the output (1–8).

1. `x = 7` then `if x > 5: print("A")` then `if x > 10: print("B")`
2. `m = 60`; the grade chain (70/60/50) — which grade?
3. `s = 0` then `for i in range(4): s += i` then `print(s)`
4. `for c in "PK": print(c)`
5. `stock = 2`; `while stock > 0: print("give"); stock -= 1`
6. `for i in range(6): if i == 3: break` then `print(i)` (outside the loop)
7. `for i in range(4): if i % 2 == 0: continue` then `print(i)` (outside)
8. `print(nothing())` where `def nothing(): x = 1` — output?

Short answers (9–14).

9. State the three syntax rules of an `if` block.
10. Explain first-True-wins with the buggy grade chain as evidence.
11. Why must every `while` body contain an update line?
12. break vs continue — one sentence and one pharmacy example each.
13. What are the four argument types? Give a one-line example of each.
14. Why does the sentinel loop use `while True` instead of a condition?

Write programs (15–20).

15. Temperature classifier: <36 hypothermia, 36–37.5 normal, >37.5 fever (chain).
16. Stock gate: `if qty >= need: dispense` else print shortfall.
17. Count fevers in `[36.8, 38.1, 37.9, 36.5, 38.4]` with a for + if.

18. Days supply for 45 tablets at 3 per day (while loop).
19. Sentinel dose-total loop that also counts entries.
20. `def bsa(height_cm, weight)` returning Mosteller BSA; call it for 170 cm, 70 kg (expect 1.82).

Answers to selected exercises

1. `A` only. 2. First Class. 3. `6` (0+1+2+3). 4. `P` then `K`. 5. `give give`. 6. `3`. 7. `3`. 8. `None`. 10. 88 caught by `>= 50` first, deeper branches unreachable. 11. Otherwise the condition never changes — infinite loop. 14. Because the exit condition is *inside* the body (the sentinel check) rather than in the `while` line. 17. 3 fevers. 18. 15 days, 0 left. 20. `import math` inside or `** 0.5`; `math.sqrt(170*70/3600)` → 1.82.

Important points for examination

1. A conditional runs a block only when its condition holds; Python marks blocks by **indentation**, not braces — a wrong indent is an `IndentationError`, and four spaces per level is the convention.
2. `else` takes no condition and is the catch-all; `elif` is tested only when every earlier test was False.
3. The chain is greedy from the top: place the **strictest condition first** or loose tests swallow the rest — with no error message.
4. Boundaries are policy: `>=` admits the limit value, `>` rejects it.
5. Nesting is one decision inside another; use it when inner questions only make sense after outer ones, and keep three levels as the ceiling.
6. `for` suits a known collection or count; `while` suits an open-ended condition, checked before each pass, updated inside each pass.
7. An forgotten update is the infinite loop; stop it with Ctrl+C, fix it with an update line.
8. `break` leaves the whole loop; `continue` abandons one round only; the sentinel loop is `while True` plus `break` on a magic word.
9. `range()` starts at 0 and excludes its stop; `range(1, 6)` yields 1–5.
10. A function is defined with `def`, runs when called, receives arguments in its parameters, and hands back a value with `return` — or `None` if no return executes.
11. `print()` displays; `return` delivers for reuse. Argument kinds: positional, keyword, default, variable-length (`*args`).
12. Modular style: one job per function (`get_dose_rate`, `calculate_dose`, `classify_bmi`), assembled by a caller — the dosage and BMI programs of this chapter are the models.

Sources for this unit: course decks Classes 6–7 and the classroom programs (`pass_fail_checker.py`, ward monitor); topic coverage cross-checked against Noteskarts *Computer Applications in Pharmacy — Unit 2* notes, MCA R24 lecture notes Unit II (functions), Yamuna Devi, Jeyabharathi & Vajpayee *Python Programming* (GCS), MRCET R17 notes, and the curated learn.biodockify.com catalog (trekhleb/learn-python, exercism/python, gto76/python-cheatsheet, W3Schools Python If...Else / For Loops). All prose and examples are original; code outputs verified by execution; originality audited against every consulted source (zero 7-word overlaps). Full credits: see the book's Sources and Acknowledgements page.